SPRINT 6 · DATOS LISTOS PARA MODELAR · S06.3

scikit-learn: métricas y validación

Un modelo que «acierta el 90 %» puede ser inútil o, peor, engañoso. Hay que medirlo bien y contra una referencia.

La API de scikit-learn y el *baseline*

```
logistica = Pipeline([("prep", hacer_preprocesado()),
                      ("clf", LogisticRegression(max_iter=1000))])   # Ejercicio 1
logistica.fit(X_train, y_train)          # aprende
pred = logistica.predict(X_test)         # predice
proba = logistica.predict_proba(X_test)[:, 1]
```

- Todos los modelos siguen el patrón `fit` / `predict`.
- **Baseline**: un modelo «tonto» (`DummyClassifier`, siempre la clase más frecuente). Si tu modelo no lo bate, no sirve.

Métricas de clasificación

Con `positivo = cancelada`:

Métrica	Pregunta que responde
Accuracy	¿Qué proporción acierta? (engaña si hay desbalance)
Precisión	De las que digo «cancelará», ¿cuántas lo hacen?
Recall	De las que cancelan, ¿cuántas detecto?
F1	Equilibrio entre precisión y recall
AUC-ROC	¿Ordena bien las reservas por riesgo?

```
confusion_matrix(y_test, pred); roc_auc_score(y_test, proba)
```

Métricas de regresión (Ejercicio 2)

```
# objetivo: precio_noche, con tipo_habitacion y mes_llegada
modelo = Pipeline([("prep", hacer_preprocesado(num=["mes_llegada"], cat=["tipo_habitacion"])),
                   ("lr", LinearRegression())]).fit(X_train, y_train)
mae  = mean_absolute_error(y_test, pred)                # error medio en euros
rmse = mean_squared_error(y_test, pred) ** 0.5          # castiga errores grandes
r2   = r2_score(y_test, pred)                           # 1 = perfecto, 0 = como la media
```

- **MAE**: se lee en las unidades de la variable («nos equivocamos 12 €»).
- Compara siempre con predecir **la media**.

Sobreajuste y sesgo–varianza (Ejercicio 3)

Un modelo muy flexible **memoriza** el entrenamiento (sobreajuste); uno demasiado simple no aprende lo importante (infraajuste).

```
for prof in profundidades:
    arbol = DecisionTreeClassifier(max_depth=prof).fit(X_train_p, y_train)
    acc_te.append(arbol.score(X_test_p, y_test))
mejor_profundidad = profundidades[int(np.argmax(acc_te))]
```

Se ve en la curva: el acierto en entrenamiento sigue subiendo, el de prueba sube y **luego baja**. El punto bueno está en medio.

Validación cruzada (Ejercicio 4)

```
auc_cv = cross_val_score(logistica, X_train, y_train,
                         cv=StratifiedKFold(5, shuffle=True, random_state=SEMILLA),
                         scoring="roc_auc")
auc_cv.mean(), auc_cv.std()
```

- Una única partición depende de la suerte; con 5 trozos obtienes **5 notas** y su variabilidad.
- Se hace con el **Pipeline completo** para que el preprocesado no filtre información.
- El test final se usa **una sola vez**, al terminar.
- Mini-reto: tabla `resultados` con `auc_cv_media` y `auc_cv_desv` del *baseline* (`DummyClassifier(strategy="prior")`) y de la logística, y una frase `conclusion`: ¿compensa el modelo?

Fuga de datos (*data leakage*, Ejercicio 5)

> Hay fuga cuando el modelo usa información que **no tendría en el momento de predecir**.

Columna candidata	¿Se conoce al reservar?
`antelacion_dias`	Sí: días entre reservar y llegar
`tarifa`	Sí: se elige al reservar
`cancelaciones_previas`	Sí: historial del cliente
`importe_devuelto`	**No**: euros reembolsados tras cancelar → **fuga**
`fecha_cancelacion`	**No**: vacía si no canceló → **fuga**

En el ejercicio rellenas `columnas_con_fuga`. Un AUC «demasiado bueno» es la primera señal de fuga.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S06_04_metricas_validacion

45 min

Baseline, métricas, validación cruzada y fuga de datos

sprints/S06/notebooks/S06_04_metricas_validacion.ipynb

RETO

Reto del sprint

20 min

Informe, README y ensayo (paso 5)

sprints/S06/guia/Reto_Sprint_06.md